

Chương 3: Quản lý bộ nhớ

Hệ điều hành

ThS. Đinh Xuân Trường

truongdx@ptit.edu.vn



Posts and Telecommunications
Institute of Technology
Faculty of Information Technology 1



CNTT1

Học viện Công nghệ Bưu chính Viễn thông

August 15, 2022

Địa chỉ và các vấn đề liên quan

Vấn đề gán địa chỉ

Địa chỉ logic và địa chỉ vật lý

Một số cách tổ chức chương trình

Tải trong quá trình chạy

Liên kết động và thư viện dùng chung

Các yêu cầu quản lý bộ nhớ

Cấp phát lại

Bảo vệ

Chia sẻ

Cấu trúc logic và cấu trúc vật lý

Phân chương bộ nhớ

Phân chương cố định

1. Địa chỉ và các vấn đề liên quan
2. Một số cách tổ chức chương trình
3. Các yêu cầu quản lý bộ nhớ
4. Phân chương bộ nhớ
5. Phân trang bộ nhớ
6. Phân đoạn bộ nhớ
7. Bộ nhớ ảo

Bộ nhớ là tài nguyên quan trọng thứ hai sau CPU trong hệ thống máy tính, bao gồm các bytes và các từ nhớ được đánh địa chỉ.

- ▶ Bộ nhớ là nơi chứa tiến trình và dữ liệu cho tiến trình
- ▶ Quản lý bộ nhớ ảnh hưởng tới tốc độ xử lý và khả năng tính toán của toàn bộ hệ thống
- ▶ Các công việc liên quan tới quản lý bộ nhớ bao gồm:
 - Quản lý bộ nhớ trống
 - Cấp phát và giải phóng bộ nhớ cho các tiến trình
 - Ngăn chặn truy cập trái phép tới các vùng nhớ
 - Ánh xạ địa chỉ giữa địa chỉ logic và địa chỉ vật lý

- ▶ Bộ nhớ máy tính được đánh địa chỉ theo các bytes hoặc các từ nhớ

Dữ liệu hoặc lệnh	Địa chỉ byte (theo Hexa)
byte (8-bit)	0x0000 0000
byte	0x0000 0001
byte	0x0000 0002
byte	0x0000 0003
byte	0x0000 0004
byte	0x0000 0005
byte	0x0000 0006
byte	0x0000 0007
.	
.	
.	
byte	0xFFFF FFFB
byte	0xFFFF FFFC
byte	0xFFFF FFDD
byte	0xFFFF FFDE
byte	0xFFFF FFDF
byte	0xFFFF FFEE

2^{32} bytes

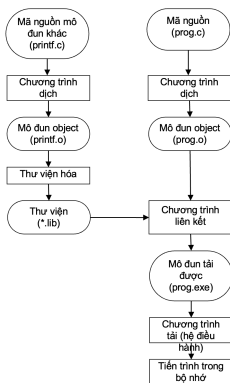
Dữ liệu hoặc lệnh	Địa chỉ word (theo Hexa)
word (32-bit)	0x0000 0000
word	0x0000 0004
word	0x0000 0008
word	0x0000 000C
word	0x0000 0010
word	0x0000 0014
word	0x0000 0018
.	
.	
.	
word	0xFFFF FFF4
word	0xFFFF FFF8
word	0xFFFF FFFC

2^{30} words

Computer Architecture

- ▶ Quá trình thực hiện tiến trình CPU đọc lần lượt các lệnh từ trong bộ nhớ và thực hiện các lệnh này
- ▶ Dữ liệu và lệnh đều được đánh địa
- ▶ CPU sẽ sử dụng địa chỉ để xác định lệnh và dữ liệu cụ thể

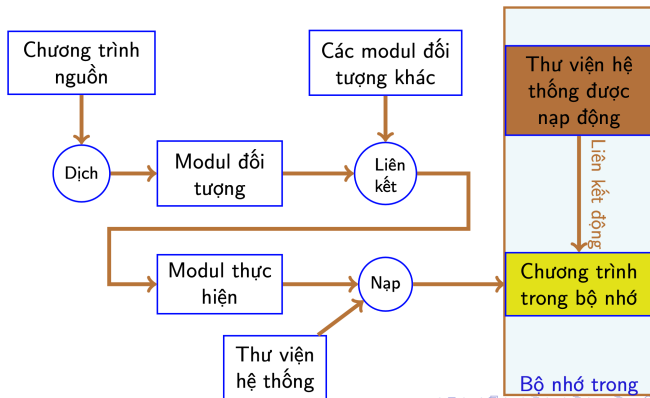
- ▶ Chương trình thường không được viết bằng ngôn ngữ máy
- ▶ Các chương trình phải trải qua một quá trình dịch và chương trình liên kết trước khi trở thành chương trình có thể tải vào và thực hiện



Địa chỉ và các vấn đề liên quan (cont.)

Vấn đề gắn địa chỉ

- ▶ Chương trình thường không được viết bằng ngôn ngữ máy
- ▶ Các chương trình phải trải qua một quá trình dịch và chương trình liên kết trước khi trở thành chương trình có thể tải vào và thực hiện



Địa chỉ và các vấn đề liên quan (cont.)

Vấn đề gán địa chỉ



- ▶ Chương trình sử dụng địa chỉ dưới dạng tên (biến, hàm)
- ▶ Khi dịch, chương trình dịch ánh xạ các tên theo địa chỉ tương đối tính từ đầu file obj
- ▶ Chương trình liên kết ánh xạ địa chỉ thành địa chỉ tương đối tính từ đầu chương trình

The screenshot shows a C++ IDE with two panes. The left pane displays the source code for a function named `square()`. The code defines an array `A` of size 5, initializes it with values {1, 4, 2}, and then calculates `B[2] = C[4]`, `C[5] = B[2] + 1`, and `A[1] = B[0] + C[3]`. The right pane shows the corresponding assembly code generated by the compiler (gcc 12.2). The assembly code uses registers like `rbp`, `rsp`, `edi`, `eax`, and `edx` to perform the operations, with addresses relative to the current instruction or frame base.

```
// Type your code here, or load an example.
void square(){
    int A[5];
    int B[3] = {1, 4, 2};
    int C[5] = {2, 3, 4, 5, 6};
    A[0] = 1;
    A[2] = 3;
    if(A[0] > C[4])
    {
        B[2] = C[4];
        C[5] = B[2] + 1;
        A[1] = B[0] + C[3];
    }
}
```

```
1 square():
2     push    rbp
3     mov     rbp, rsp
4     mov     DWORD PTR [rbp-20], 1
5     mov     DWORD PTR [rbp-16], 4
6     mov     DWORD PTR [rbp-12], 2
7     mov     DWORD PTR [rbp-8], 2
8     mov     DWORD PTR [rbp-4], 2
9     mov     DWORD PTR [rbp-40], 4
10    mov     DWORD PTR [rbp-36], 5
11    mov     DWORD PTR [rbp-32], 6
12    mov     DWORD PTR [rbp-8], 1
13    mov     DWORD PTR [rbp+8], 3
14    mov     edi, DWORD PTR [rbp-8]
15    mov     eax, DWORD PTR [rbp-32]
16    cmp     edi, eax
17    jle     .L3
18    mov     eax, DWORD PTR [rbp-32]
19    mov     DWORD PTR [rbp+12], eax
20    mov     eax, DWORD PTR [rbp-12]
21    add     eax, 1
22    mov     DWORD PTR [rbp-28], eax
23    mov     edi, DWORD PTR [rbp-20]
24    mov     eax, DWORD PTR [rbp-36]
25    add     eax, edx
26    mov     DWORD PTR [rbp-4], eax
27
28 .L3:
29    pop     rbp
30    ret
```

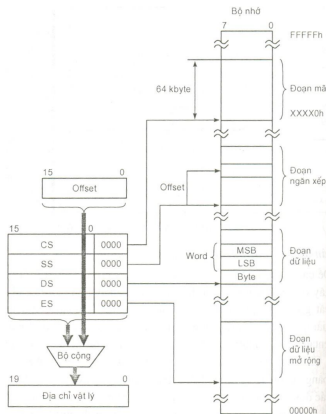

► Quản lý bộ nhớ:

- **Cấp phát bộ nhớ** cho các tiến trình
- **Tái định vị**, ánh xạ địa chỉ lệnh và dữ liệu từ các tiến trình vào trong bộ nhớ
- **Bảo vệ tiến trình**: phần nào bộ nhớ cho tiến trình nào thì tiến trình khác không được sử dụng
- **Chia sẻ**: các tiến trình có thể chia sẻ các không gian nhớ khi tương tác với nhau

Địa chỉ và các vấn đề liên quan

Địa chỉ logic và địa chỉ vật lý

VXL 8086 dùng 20 bit mã hoá địa chỉ 1MB từ 00000h - FFFFFh



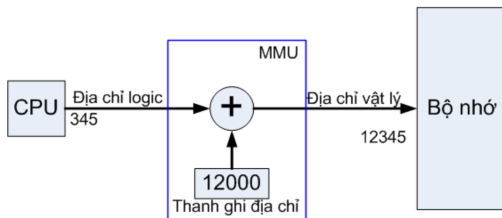
Địa chỉ vật lý 20-bit của một ô nhớ = địa chỉ đoạn 16-bit được dịch trái 4 bit (nhân với 16) + địa chỉ lệnh 16-bit.

Địa chỉ và các vấn đề liên quan (cont.)

Địa chỉ logic và địa chỉ vật lý

► Địa chỉ logic

- Gán cho các lệnh và dữ liệu không phụ thuộc vào vị trí cụ thể tiến trình trong bộ nhớ. Được sinh ra trong tiến trình (CPU đưa ra)
- CTƯD chỉ quan tâm tới địa chỉ logic còn địa chỉ vật lý là do HĐH
- Khi thực hiện chương trình, CPU “nhìn thấy” và sử dụng địa chỉ logic này để trỏ đến các phần khác nhau của lệnh, dữ liệu
- Để truy cập bộ nhớ, địa chỉ logic cần được ánh xạ thành địa chỉ vật lý và được khối quản lý bộ nhớ (MMU) chuyển sang địa chỉ vật lý khi truy nhập tới đối tượng trong chương trình



Ví dụ: JMP 010A: Nhảy tới ô nhớ có vị trí 010Ah tại cùng đoạn mã lệnh (CS - Code Segment)

Nếu CS = 1555h, sẽ đi tới vị trí: $1555h * 10h + 010Ah = 1560Ah$

▶ Địa chỉ vật lý:

- Là địa chỉ chính xác trong bộ nhớ máy tính
- Các mạch nhớ sử dụng để truy nhập tới chương trình và dữ liệu
- ▶ Địa chỉ logic được chuyển thành địa chỉ vật lý nhờ khối ánh xạ địa chỉ. (MMU=Memory Mapping Unit)

Các vấn đề quan trọng trong tổ chức chương trình và quản lý bộ nhớ:

- ▶ Giảm không gian chương trình chiếm trên đĩa, trên bộ nhớ
- ▶ Sử dụng không gian nhớ hiệu quả
- ▶ Các kỹ thuật tổ chức và sử dụng bộ nhớ hiệu quả:
 - Tải trong quá trình chạy
 - Liên kết động và thư viện dùng chung

Một số cách tổ chức chương trình

Tải trong quá trình chạy



Toàn bộ chương trình thông thường được tải vào bộ nhớ để thực hiện.

Đối với chương trình lớn, trong một phiên làm việc, một số phần của chương trình có thể không dùng tới. Các hàm này sẽ chiếm vô ích trong bộ nhớ, đồng thời tăng thời gian tải chương trình lúc đầu.

Giải pháp:

- ▶ Hàm chưa bị gọi thì chưa tải vào bộ nhớ
- ▶ Chương trình chính được load vào bộ nhớ và chạy
- ▶ Khi có lời gọi hàm:
 - Chương trình sẽ kiểm tra hàm đó được tải vào chưa.
 - Nếu chưa, chương trình sẽ tiến hành tải sau đó ánh xạ địa chỉ hàm vào không gian chung của chương trình và thay đổi bảng địa chỉ để ghi lại các ánh xạ đó
- ▶ Lập trình viên đảm nhiệm kiểm tra và tải hàm, HĐH chỉ cung cấp các hàm thư viện cho module

Một số cách tổ chức chương trình

Liên kết động và thư viện dùng chung



Trong quá trình liên kết tĩnh các hàm và thư viện được liên kết luôn vào mã chương trình

Kích thước chương trình = Kích thước chương trình vừa được dịch + Kích thước các hàm thư viện

▶ Các hàm sẽ có mặt lặp đi lặp lại trong các chương trình

▶ Lãng phí không gian cả trên đĩa và bộ nhớ trong

Giải quyết sử dụng **kỹ thuật liên kết động** : Trong giai đoạn liên kết, không kết nối các hàm thư viện vào chương trình mà chỉ chèn các thông tin về hàm thư viện đó (stub).

Một số cách tổ chức chương trình (cont.)

Liên kết động và thư viện dùng chung



► Các module thư viện được liên kết trong quá trình thực hiện:

- Không giữ bản sao các module thư viện mà tiến trình giữ đoạn mã nhỏ chứa thông tin về modul thư viện
- Khi đoạn mã nhỏ được gọi, nó kiểm tra module tương ứng đã có trong bộ nhớ chưa. Nếu chưa, thì tải phần còn lại và dùng.
- Lần tiếp theo cần sử dụng, modul thư viện sẽ được chạy trực tiếp
- Mỗi module thư viện chỉ có 1 bản sao duy nhất chứa trong MEM
- Cần hỗ trợ từ HĐH

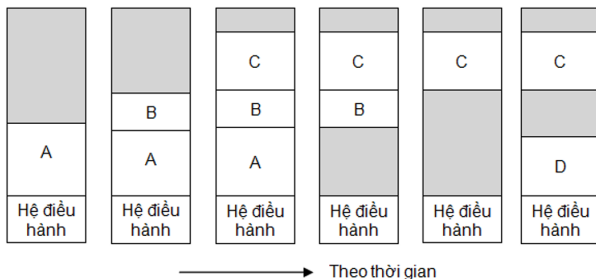
Các yêu cầu liên quan đến việc quản lý bộ nhớ bao gồm:

- ▶ **Cấp phát lại** cho các tiến trình
- ▶ **Bảo vệ**: phần nào bộ nhớ cho tiến trình nào thì tiến trình khác không được sử dụng
- ▶ **Chia sẻ**: các tiến trình có thể chia sẻ các không gian nhớ khi tương tác với nhau
- ▶ **Cấu trúc**: cấu trúc logic và cấu trúc vật lý

Các yêu cầu quản lý bộ nhớ

Cấp phát lại

- ▶ Cần có khả năng trao đổi các tiến trình vào và ra ngoài MEM để tối đa sử dụng vi xử lý
- ▶ Không thể yêu cầu tiến trình được chuyển lại vào MEM thì phải vào đúng chỗ nó đã dùng trước khi bị chuyển ra



- ▶ Mỗi tiến trình phải được bảo vệ khỏi các tham chiếu không mong muốn từ các tiến trình khác vào vùng nhớ dành cho mình
- ▶ Khi 1 vùng nhớ đã dùng cho tiến trình này thì nó không cho phép tiến trình khác tham chiếu vào vùng nhớ đang dùng đó.
- ▶ Mọi tham chiếu bộ nhớ của 1 tiến trình phải được kiểm tra lúc chạy
- ▶ HĐH không đoán trước được mọi tham chiếu MEM nên phần cứng vi xử lý đảm nhiệm

- ▶ Nhiều tiến trình cần và được phép truy cập vào cùng 1 vùng nhớ
- ▶ Các tiến trình đang cộng tác cần chia sẻ truy nhập tới 1 cấu trúc dữ liệu
- ▶ Cho phép truy cập tới các vùng chia sẻ

Cấu trúc logic:

- ▶ MEM được cấu trúc 1 cách tuyến tính gồm các byte, còn chương trình được tổ chức thành các modul
- ▶ Phải đáp ứng để:
 - Các module có thể được viết và thông dịch 1 cách độc lập
 - Mức độ bảo vệ có thể khác nhau
 - Module có thể được chia sẻ giữa các tiến trình

Cấu trúc vật lý:

- ▶ Cấu trúc vật lý 2 mức:
 - Bộ nhớ chính: nhanh; chi phí cao, dung lượng ít
 - Bộ nhớ phụ: dung lượng lớn, cho phép lưu chương trình và dữ liệu trong thời gian dài
- ▶ Hệ thống có trách nhiệm chuyển đổi thông tin giữa 2 mức

- ▶ Để thực hiện tiến trình, HĐH cần cấp phát cho tiến trình không gian nhớ cần thiết.
- ▶ Việc cấp phát và quản lý vùng nhớ là chức năng quan trọng của HĐH
- ▶ Một kỹ thuật cấp phát đơn giản nhất là mỗi tiến trình được cấp một vùng bộ nhớ liên tục
- ▶ HĐH tiến hành chia bộ nhớ thành các phần liên tục là chương (partition), mỗi tiến trình sẽ được cung cấp một chương để chứa lệnh và dữ liệu của mình.
- ▶ Quá trình phân chia bộ nhớ thành chương như vậy gọi là *phân chương bộ nhớ*.
- ▶ Tùy thuộc việc lựa chọn vị trí và kích thước của chương, có thể phân biệt phân chương cố định và phân chương động

Các chiến lược quản lý bộ nhớ:

- ▶ Phân chương cố định
- ▶ Phân chương động
- ▶ Phân trang
- ▶ Phân đoạn
- ▶ Kết hợp phân đoạn - phân trang

Nguyên tắc

- ▶ Bộ nhớ được chia thành n phần
 - Mỗi phần được gọi là một chương (partition)
 - Mỗi chương ở một vị trí cố định
 - Chương được sử dụng như một vùng nhớ độc lập
 - ▶ Mỗi chương chứa được đúng một tiến trình
 - ▶ Khi được tải vào, tiến trình được cấp phát một chương. Sau khi tiến trình kết thúc, HĐH giải phóng chương và chương có thể được cấp phát cho tiến trình mới.
- Chương có thể có kích thước bằng nhau hoặc khác nhau

Phân chương bộ nhớ (cont.)

Phân chương cố định

Ví dụ: Xét hệ thống dưới đây bộ nhớ được tổ chức theo phân chương cố định, tính thời gian HĐH thực hiện xong các tiến trình sau:



Process	Size	time
P_1	120	20
P_2	80	15
P_3	70	5
P_4	50	5
P_5	140	12
Hàng đợi		

► Kích thước các chương bằng nhau:

- Ưu điểm: Đơn giản
- Nhược điểm: Kích thước chương trình $>$ kích thước chương dẫn đến không thể cấp phát
- Gây phân mảnh trong

► Kích thước các chương khác nhau:

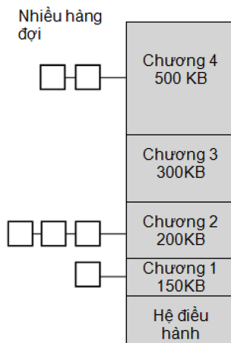
- Có hai cách lựa chọn chương nhớ để cấp cho tiến trình đang chờ đợi bằng cách chọn chương có kích thước nhỏ nhất
 - Mỗi chương có một hàng đợi riêng
 - Một hàng đợi chung cho tất cả các chương

Phân chương bộ nhớ (cont.)

Phân chương cố định



Mỗi chương có một hàng đợi riêng: *tiến trình có kích thước phù hợp với chương nào sẽ nằm trong hàng đợi của chương đó*

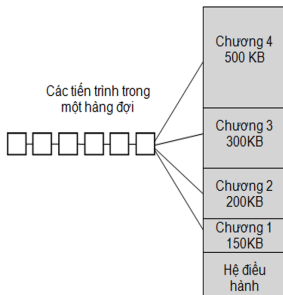


- Ưu điểm: Tiết kiệm bộ nhớ, giảm phân mảnh trong
- Nhược điểm: Hệ thống không tối ưu, có thời điểm hàng đợi chương lớn thì rỗng, hàng đợi chương nhỏ hơn chứa nhiều tiến trình.

Phân chương bộ nhớ (cont.)

Phân chương cố định

Một hàng đợi chung cho tất cả các chương: *Mỗi khi có một chương trống tiến trình nằm gần đầu hàng đợi nhất và có kích thước phù hợp với chương nhất sẽ được tải và thực hiện*



- Ưu điểm: Tiết kiệm bộ nhớ, giảm phân mảnh trong và tối ưu hệ thống

Nhận xét về phân chương cố định

► Ưu điểm

- Đơn giản và ít xử lý
- Giảm thời gian tìm kiếm

► Nhược điểm

- Hệ số song song không thể vượt quá số lượng chương của bộ nhớ
- Bị phân đoạn bộ nhớ
 - Kích thước chương trình lớn hơn kích thước chương lớn nhất
 - Tổng bộ nhớ tự do còn lớn, nhưng không dùng để nạp các chương trình khác

Chương 3 Quản lý bộ nhớ

- ▶ Địa chỉ và các vấn đề liên quan
- ▶ Một số cách tổ chức chương trình
- ▶ Các yêu cầu quản lý bộ nhớ

Chương 3 Quản lý bộ nhớ

- ▶ Phân chương bộ nhớ
- ▶ Phân trang bộ nhớ
- ▶ Phân đoạn bộ nhớ
- ▶ Bộ nhớ ảo